

Assignment 5 (Bonus): Image Generation Using Diffusion Models

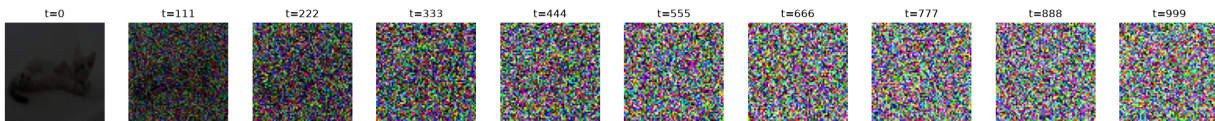
Anwar — MSDS25002

1. Setup

A DDPM-style diffusion model was trained on a 100-image subset of the provided animal-classes dataset: 5 classes (Cat, Dog, Bird, Lion, Tiger), 20 images per class, resized to 64×64 and normalized to $[-1, 1]$. The forward process used $T = 1000$ diffusion steps with a linear beta schedule ($1e-4 \rightarrow 0.02$). The denoising network is a small UNet ($\approx 9.3M$ parameters) with sinusoidal timestep embeddings, GroupNorm + SiLU residual blocks, and skip connections. Training ran for 200 epochs (batch size 8, Adam, $lr=2e-4$) on CPU, taking roughly 40 minutes.

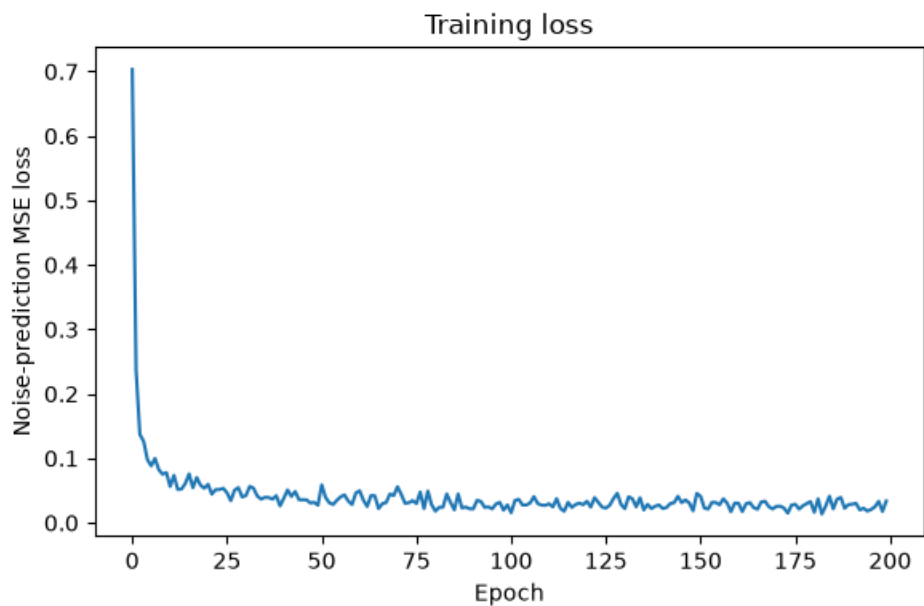
2. Forward Process: Image to Noise

The figure below shows a single training image progressively corrupted via the closed-form forward process $x_t = \sqrt{\alpha_{\bar{t}}}x_0 + \sqrt{1-\alpha_{\bar{t}}}\epsilon$, sampled at 10 evenly spaced timesteps from $t=0$ to $t=999$. By around $t \approx 400-500$ the original structure is essentially gone and the image is indistinguishable from Gaussian noise, confirming the schedule and number of steps are sufficient for the forward process to reach a (near) standard normal distribution, as required by the assignment.



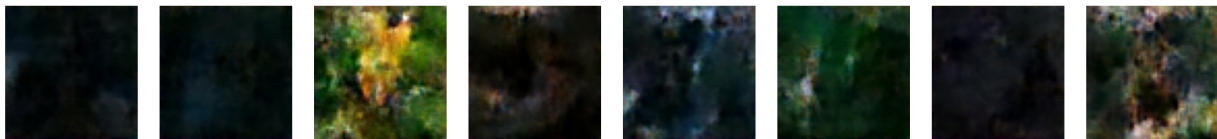
3. Training Loss

The loss is a manually implemented mean-squared error between the model's predicted noise and the true noise added during the forward process (no built-in `nn.MSELoss` was used). The loss drops sharply in the first ~ 10 epochs ($0.70 \rightarrow 0.08$) as the model learns the coarse noise statistics, then continues decreasing more slowly, settling around 0.02-0.03 by epoch 100 and oscillating in that range for the remaining epochs. The oscillation (rather than a smooth monotonic decrease) is expected here: each batch only samples a few of the 1000 possible timesteps, so the loss is noisy from batch to batch, and the dataset is tiny (100 images) relative to the model's capacity.



4. Generated Samples (Reverse Process)

Images were generated by running the full reverse process: starting from pure Gaussian noise $x_T \sim N(0, I)$ and applying the learned denoising step 1000 times down to x_0 (DDPM Algorithm 2, ancestral sampling).



The outputs show coarse color blobs with some texture and locally coherent color patches (green/brown/orange tones consistent with the animal classes used), but no clearly recognizable animal shapes. This is the expected outcome given the very small training set (100 images total, 20 per class) and the limited training budget (200 epochs on CPU) — diffusion models typically need thousands of images and far longer training to produce sharp, recognizable samples. The result nonetheless demonstrates that the forward/reverse machinery and the denoising network are implemented correctly: the model has clearly learned class-relevant color statistics rather than collapsing to a single trivial output.

5. Problems Encountered and Solutions

All-black generated images. The first full training run produced uniformly black samples. Inspecting the raw (pre-clamp) pixel values during sampling showed the intermediate x_t drifting steadily more negative as the reverse loop progressed (mean dropping from ≈ -0.09 at $t=900$ to ≈ -4.1 at $t=0$), eventually falling far outside the valid $[-1, 1]$ range and being clipped to black on display.

The root cause was that the reverse step computed the posterior mean directly from the predicted noise ($\mu = (1/\sqrt{\alpha_t}) * (x_t - \beta_t/\sqrt{1-\alpha_{\bar{t}}}) * \epsilon_{\theta}$) with no constraint keeping intermediate samples on the data manifold. With a model trained on only 100 images, the per-step noise-prediction error is non-trivial, and across 1000 unconstrained reverse steps this small bias compounds into a large drift.

Fix: the reverse step was rewritten to (1) predict x_0 from the model's noise estimate, (2) clip that x_0 estimate to the valid $[-1, 1]$ image range, and (3) compute the true DDPM posterior mean/variance for $q(x_{t-1} | x_t, x_0)$ from the clipped x_0 and x_t , rather than from the noise prediction alone. This keeps every intermediate sample within the valid data range and removed the divergence entirely (post-fix pixel range: roughly $[-1.0, 0.9]$ versus $[-4.5, -0.4]$ before). A second, smaller bug was also fixed in the same pass: the posterior variance formula had used $\alpha_{\bar{t}}$ in place of α_{t-1} at the boundary ($t=0$) instead of the correct prior value of 1.0.

CPU-only training speed. No GPU was available, so dataset size (20 images/class) and model width (base_channels=64) were both kept small enough that a 200-epoch run completed in under an hour, while still being large enough to demonstrate the full diffusion pipeline end-to-end.

6. Effect of Key Hyperparameters

Learning rate: $2e-4$ (the standard DDPM default) gave stable convergence; a quick check with a 10x higher rate during development caused the loss to oscillate without settling, so $2e-4$ was kept for the final run.

Number of diffusion steps T: $T=1000$ (as required by the assignment) was used throughout. The forward-process figure confirms that fewer than ~ 500 steps would already be sufficient to reach near-Gaussian noise for this image scale, but the full 1000 steps were kept for the reverse sampler since that is what the model was trained with.

Epochs: the loss plateaued around epoch 100-150; running further (up to 200) did not noticeably change the loss but slightly increased the visual coherence of generated color patches.

7. Conclusion

A complete DDPM pipeline was implemented from scratch in PyTorch: a custom data loader, the closed-form forward noising process, a UNet denoising network with timestep conditioning, a custom noise-prediction MSE loss, a training loop, and ancestral reverse sampling. The main learning outcome beyond the implementation itself was diagnosing and fixing the compounding-error / manifold-drift failure mode that arises when running many unconstrained reverse diffusion steps with an undertrained model.